

HTL Maturatrainer

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für Medientechnik

Eingereicht von:

Thomas Gossenreiter
Stefanie Steinmair

Betreuer:

Birgit Schröder

Leonding, 6. April 2026

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 6. April 2026

Thomas Gossenreiter & Stefanie Steinmair

Abstract

Brief summary of our amazing work. In English. This is the only time we have to include a picture within the text. The picture should somehow represent your thesis. This is untypical for scientific work but required by the powers that are. Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.



Zusammenfassung

Zusammenfassung unserer genialen Arbeit. Auf Deutsch. Das ist das einzige Mal, dass eine Grafik in den Textfluss eingebunden wird. Die gewählte Grafik soll irgendwie eure Arbeit repräsentieren. Das ist ungewöhnlich für eine wissenschaftliche Arbeit aber eine Anforderung der Obrigkeit. *Bitte auf keinen Fall mit der Zusammenfassung verwechseln, die den Abschluss der Arbeit bildet!* Suspendisse vel felis.

Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.



Inhaltsverzeichnis

1	Einleitung	1
1.1	Ausgangslage	1
1.2	Motivation und Problemstellung	1
1.3	Grundlagen und Analyse	2
1.3.1	Marktanalyse	2
	StudySmarter	2
1.3.2	Maturatraining digitalisieren	2
1.4	Lernstrategien	2
1.4.1	Leitner-System	2
1.4.2	Spaced Repetition	2
1.4.3	Vergleich verschiedener Lernstrategien und Schlussfolgerung . .	2
1.5	Zielsetzung	3
2	Technologische Entscheidungen und Architektur	4
2.1	Backend	4
2.1.1	Backend-Technologien im Vergleich	5
	Quarkus	5
	Springboot	7
	Node.js	9
2.1.2	Auswahl der Backend-Technologie für die Lernplattform	11
2.2	Frontend	13
2.2.1	Angular	13
2.2.2	Flutter	13
2.2.3	Ergebnis	13
3	Praktische Umsetzung	14
3.1	Usecases	14
3.1.1	User Flow	14

3.1.2	Übungsmodus	15
3.1.3	Prüfungssimulationen	16
3.1.4	Fortschrittsübersicht	17
3.2	Use Case: Üben	17
3.2.1	Verwaltung von Fragenpools	18
3.2.2	Fragenhandling und Fragetypen	18
3.2.3	Übungsmodus mit direktem Feedback	21
3.2.4	Lösungswege und Nutzerinteraktion	21
3.3	Use Case: Prüfungssimulation	21
3.3.1	Prüfungsmodus mit Zeitlimit	21
3.3.2	Bewertung und Ergebnisausgabe	21
3.4	Use Case: Fortschritt einsehen & analysieren	21
3.4.1	Lernstrategien und Fehleranalyse	21
3.4.2	Spaced Repetition nach Leitner	22
3.4.3	Lernstatistiken und Visualisierung	23
3.5	Usability-Test mit Mitschülern	23
3.6	Implementierungsdetails	23
3.6.1	Datenmodell	23
3.6.2	Überblick über Komponenten	23
3.6.3	Custom Css-Klassen und eingebundene Bibliotheken	23
4	Zusammenfassung	24
	Literaturverzeichnis	VI
	Abbildungsverzeichnis	VII
	Tabellenverzeichnis	VIII
	Quellcodeverzeichnis	IX
	Anhang	X

1 Einleitung

1.1 Ausgangslage

Die Vorbereitung auf Tests, Schularbeiten und schließlich die Matura erfordern ein hohes Maß an Eigenverantwortung und Organisation. Im Moment erfolgt das Lernen größtenteils mithilfe von Skripten, Arbeitsblättern und Übungsbeispielen aus dem Unterricht. Dabei fehlt allerdings die Möglichkeit, den eigenen Wissensstand interaktiv zu überprüfen oder seine eigenen Fehler gezielt zu analysieren. Somit bleibt häufig unklar, ob die gelernten Inhalte auch wirklich verstanden wurden oder ob man sie nur oberflächlich reproduzieren konnte.

Derzeit existieren nur wenige digitale Lernangebote, die auf den schulischen Kontext der HTL anwendbar sind und dabei auf Prüfungs- und Maturasituationen vorbereiten. Viele Lernplattformen sind entweder zu allgemein gehalten oder dienen zur reinen Stoffvermittlung, ohne dabei den Lernprozess aktiv zu unterstützen. Bei vorhandenen Lernmethoden gibt es außerdem meist kein unmittelbares Feedback auf Antworten. Schüler:innen wissen daher oft nicht, warum eine Lösung richtig oder falsch ist und können ihre Fehler nur schwer nachvollziehen.

Weiters fehlt eine Möglichkeit, den individuellen Lernfortschritt strukturiert darzustellen. Falsch beantwortete Fragen gehen im Lernprozess verloren und werden häufig nicht mehr wiederholt. Lernende können somit nur schwer gezielt an ihren Schwächen arbeiten und Schwerpunkte erkennen. Ebenso gibt es kein System, das Wiederholungen von Aufgaben anhand bestehender Lernstrategien organisiert oder langfristiges Lernen unterstützt.

Diese Probleme führen bei den Schüler:innen mit der Zeit zu Unsicherheit, ineffizientem Lernen und erhöhtem Stress. Sie investieren viel Zeit ohne dabei wirklich ihren Wissensstand einschätzen zu können oder konkrete Rückmeldung über ihren Fortschritt zu erhalten.

1.2 Motivation und Problemstellung

Die im vorherigen Abschnitt beschriebene Ausgangssituation zeigt, dass der Lernprozess bei der Vorbereitung für die Matura stark von eigenständigem Üben und individueller Organisation geprägt ist. Gleichzeitig fehlen jedoch geeignete digitale Werkzeuge, die Lernenden dabei helfen, ihren Wissensstand realistisch einzuschätzen und Schwächen gezielt zu verbessern. Daraus ergibt sich eine zentrale Fragestellung: **Wie kann der Lernprozess so unterstützt werden, dass Lernen strukturierter, transparenter und nachhaltiger wird?**

Ein wesentliches Problem besteht darin, dass Fehler aktuell nur selten bewusst analysiert werden. Erhalten Lernende kein unmittelbares Feedback, so ist oft schwer zu klären, welche Denk- oder Verständnisfehler vorliegen und wie man diese in Zukunft vermeiden

kann. Dadurch wiederholen sie Inhalte oft unsystematisch, statt gezielt an den Gebieten zu arbeiten, in denen immer noch Unsicherheit vorliegt. Dies erschwert den Aufbau eines langfristig stabilen Wissensfundaments.

Hinzu kommt, dass der individuelle Lernfortschritt zu wenig sichtbar ist. Für Schüler:innen bestehen kaum Möglichkeiten, ihren aktuellen Lernfortschritt einschätzen oder Entwicklungen über einen längeren Zeitraum nachvollziehen zu können. Besonders bei der Maturavorbereitung führt dies zu einem Gefühl von mangelnder Kontrolle über den Lernprozess. Anstelle von strukturierter Vorbereitung für die Matura haben Lernende oft das Gefühl, sie würden einfach auf gut Glück lernen.

Aufgrund dieser Situation entstand die Motivation, sich im Zuge dieser Diplomarbeit mit der Frage auseinanderzusetzen, wie Lernprozesse im schulischen Rahmen gezielt unterstützt und verbessert werden können. Besonders wichtig ist dabei, wie digitale Werkzeuge einen großen Beitrag leisten können, Fehler zu identifizieren, mehr Kontrolle über den Lernprozess zu fördern und eigenständiges Lernen langfristig zu stärken.

1.3 Grundlagen und Analyse

1.3.1 Marktanalyse

Analyse bestehender Lernplattformen und digitaler Angebote.

StudySmarter

Funktionsweise, Zielgruppe, Vorteile/Schwächen, Vergleich mit eigenem Projekt.

1.3.2 Maturatraining digitalisieren

Wie war Maturatraining früher - wie muss digitalisiert werden

1.4 Lernstrategien

1.4.1 Leitner-System

Karteikarten-Prinzip, Vorteile, Umsetzung im Projekt.

1.4.2 Spaced Repetition

Verteiltes Lernen, Effektivität, Umsetzung digital.

1.4.3 Vergleich verschiedener Lernstrategien und Schlussfolgerung

Leitner, Spaced Repetition, klassisches Lernen – Vor- und Nachteile - Ergebnis.

1.5 Zielsetzung

Was soll am Ende entstehen? Funktionen und Ziele der Plattform.

2 Technologische Entscheidungen und Architektur

2.1 Backend

Das Backend bildet die zentrale Kernkomponente der Lernplattform. Es stellt die Verbindung zwischen Datenbank, Geschäftslogik und dem Frontend her und ist somit für die Verarbeitung von Anfragen von außen und die Verwaltung der persistierten Daten aus der Datenbank verantwortlich. Es dient als gemeinsame technische Grundlage beider Diplomarbeitsprojekte und verarbeitet sowohl die Funktionen des interaktiven Lernens als auch jene des Lernmaterialsystems. Alle Anfragen aus dem Frontend werden über REST-Schnittstellen an das Backend übermittelt, dort validiert, verarbeitet und anschließend persistiert oder als Antwort zurückgegeben.

Ein Aufgabenbereich des Backends liegt in der Verwaltung von lernbezogenen Inhalten. Dazu zählen unter anderem Fragen und Lösungsvorschläge, sowie von Nutzer:innen bereitgestellte Lernmaterialien. Lerninhalte, die von anderen Nutzer:innen hochgeladen oder erstellt werden, sollen im Backend gespeichert und strukturiert abgelegt werden, sodass sie von beiden Teilprojekten verwendet und wiederverwendet werden können.

Darüber hinaus übernimmt das Backend die Auswertung und Verarbeitungen von Nutzeraktionen im interaktiven Lernsystem. Dazu zählen insbesondere das Beantworten von Fragen, das Erstellen von Prüfungen und das Protokollieren von Fortschritts- und Fehlerdaten. Antworten auf Fragen werden vom Frontend ans Backend übermittelt und dort validiert. Dabei besitzt das Backend die endgültige Entscheidungshoheit über die Annahme und Interpretation von Anfragen und stellt sicher, dass keine ungültigen Datensätze ins System gelangen.

Ein wesentlicher Aufgabenbereich des Backends liegt in der Verwaltung von Lernmaterialien. Nutzer:innen sollen die Möglichkeit haben, ihre Lernmaterialien hochladen zu können. Diese sollen dann persistiert in der Datenbank liegen und später wieder abgerufen werden können. Dazu zählen sowohl Mitschriften, als auch selbst erstellte Fragen.

Durch diese Aufgabenteilung fungiert das Backend als gemeinsame, konsistente Daten- und Logikschicht der Plattform. Es ermöglicht, dass beide Diplomarbeitsprojekte unabhängig voneinander Funktionen entwickeln können, während Lerninhalte, Fragen und Fortschrittsinformationen dennoch in einem einheitlichen System verarbeitet und verwaltet werden.

Nach Ausführung der Aufgabenbereiche, welche das Backend abdecken muss, wird also ersichtlich, wie zentral die Rolle des Backends ist. Daher ist auch die Wahl einer geeigneten und verlässlichen Technologie von besonderer Bedeutung. Im folgenden Abschnitt werden also verschiedene Backend-Technologien analysiert, in Bezug auf die

Eignung für die Anforderungen dieser Diplomarbeit bewertet und anschließend die Entscheidung für das letztendlich eingesetzte Framework begründet.

2.1.1 Backend-Technologien im Vergleich

Bevor die finale Entscheidung auf Quarkus fiel, wurden verschiedene Backend-Frameworks hinsichtlich ihrer Eignung für die Lernplattform analysiert. Kriterien waren unter anderem:

- Performance und Ressourcenverbrauch
- Entwicklungsproduktivität
- Integration mit Datenbanken
- Unterstützung für REST-APIs
- Community- und Unternehmenssupport
- Skalierbarkeit und Wartbarkeit

Quarkus



Abbildung 1: Logo von Quarkus

Quarkus ist ein vergleichsweise modernes, Java-basiertes Framework, das speziell für den Einsatz in Cloud- und Containerumgebungen entwickelt wurde. Anders als bei klassischen Java-Frameworks wie Spring Boot oder Jakarta EE fokussiert sich Quarkus stark auf eine geringe Startzeit, niedrigen Speicherverbrauch sowohl auf der Festplatte, als auch im Arbeitsspeicher und die effiziente Ausführung in Mircoservice-Architekturen und serverlosen Umgebungen.

Die Basis von Quarkus bildet eine Kombination aus Jakarta EE, Eclipse MicroProfile und eine Reihe aus optimierten Bibliotheken. Ein zentrales Ziel von Quarkus besteht darin, die JVM-basierte Backend-Entwicklung an moderne Cloud-Infrastrukturen anzupassen, dabei aber nicht auf etablierte Java-Standards verzichten zu müssen. [1]

Architekturprinzip: Build-Time-Augmentation

Eine der zentralen technischen Besonderheiten von Quarkus liegt in der sogenannten *Build-Time-Augmentation*. Während klassische Frameworks viele Konfigurationen und Initialisierungsschritte erst zur Laufzeit durchführen, werden diese bei Quarkus weitgehend bereits während des Build-Prozesses ausgeführt. [2]

Dadurch ergeben sich:

- deutlich reduzierte Startzeiten
- geringere Anforderungen an die CPU und den Speicher
- weniger Reflection-Overhead

- bessere Eignung für Container-Deployments

Insbesondere bei Microservices oder bei horizontal skalierenden Anwendungen stellt dies einen großen Vorteil dar.

Unterstützte Technologien und Laufzeitkomponenten

Quarkus stellt eine breite Palette an Erweiterungen (Extensions) bereit. [3] Für typische Web- und Datenbankanwendungen besonders relevant sind:

- **RESTEasy Classic Jackson**
für performante REST-Schnittstellen
- **Hibernate ORM mit Panache**
für vereinfachte Datenbankzugriffe und Entity-Verwaltung
- **Quarkus Security & JWT-Authentifizierung**
für Zugriffsschutz und Rollenverwaltung

Diese Erweiterungen machen Quarkus besonders flexibel und eignen sich deshalb auch besonders für Anwendungen, die immer erweiterbar bleiben sollen und modular aufgebaut sind.

Entwicklungsunterstützung und Produktivität

Für die Entwicklungsumgebung bietet Quarkus einen eingebauten Dev-Modus, der automatisches Neuladen von Klassen und Konfigurationen ermöglicht. Änderungen am Quellcode werden ohne manuelles Neustarten der Anwendung übernommen.

Das führt zu verkürzten Entwicklungszyklen, da nicht nach jeder kleinen Änderung auf das erneute Starten der Anwendung gewartet werden muss. Gleichzeitig erhalten Entwickler unmittelbares Feedback und die Zeit für die Entwicklung kann effizienter genutzt werden. Der Dev-Modus unterstützt außerdem integrierte Test- und Debug Werkzeuge.

Eignung für die im Projekt entwickelte Lernplattform

Vorteile für die Lernplattform	Einschränkungen
Kurze Startzeiten und geringer Ressourcenverbrauch	Kleinere Community im Vergleich zu etablierten Frameworks
Effiziente Verarbeitung von REST-Anfragen	Weniger verfügbare Praxisbeispiele und Dokumentationen
Gute Eignung für Container- und Cloud-Umgebungen	Teilweise höherer Einarbeitungsaufwand
Modularer und erweiterbarer Systemaufbau	Native-Builds erfordern projektspezifische Prüfung

Tabelle 1: Zusammenfassung der Bewertung von Quarkus für die Lernplattform

Kriterienbewertung Quarkus

Die nachfolgende stichpunktartige Übersicht fasst die Bewertung von Quarkus anhand der definierten Vergleichskriterien zusammen:

- **Performance und Ressourcenverbrauch:** sehr gut (geringe Startzeiten, niedriger Speicherverbrauch)
- **Entwicklungsproduktivität:** gut (Dev-Modus verkürzt Entwicklungszyklen)
- **Integration mit Datenbanken:** gut (Hibernate ORM mit Panache)
- **Unterstützung für REST-APIs:** sehr gut (RESTEasy Classic Jackson)
- **Community- und Unternehmenssupport:** in Ordnung (kleinere Community, weniger Praxisbeispiele)
- **Skalierbarkeit und Wartbarkeit:** sehr gut (modular, gut für Container- und Cloud-Deployments)

Springboot



Abbildung 2: Logo von Springboot

Spring Boot ist ein Java-basiertes Framework zur schnellen Entwicklung von produktionsreifen Anwendungen. Es ist Teil des Spring-Ökosystems und baut somit direkt auf dem etablierten Spring-Framework auf, jedoch mit dem sogenannten "*Konvention über Konfiguration*" - Prinzip, dem Kernprinzip von Spring Boot. Ziel dieses Prinzips ist es, den Entwicklungsaufwand durch sinnvolle Standardwerte und automatische Konfigurationen zu mindern. Entwickler:innen müssen also weniger Entscheidungen treffen und können sich direkt der Geschäftslogik widmen. Spring Boot wird häufig für Web-APIs, Micorservices und komplexe Enterprise-Anwendungen eingesetzt. [4]

Architekturprinzip: Konvention über Konfiguration

Spring Boot zeichnet sich vor allem dadurch aus, möglichst viele Einstellungen und Abhängigkeiten automatisch zu konfigurieren. Sobald entsprechende Bibliotheken ins Projekt eingebaut werden, erkennt Spring Boot deren Bedarf nimmt passende Standard-Konfigurationen zur Laufzeit vor. Dadurch soll der Initialaufwand minimiert werden.

Wichtige Eigenschaften dieses Prinzips sind:

- Automatische Konfiguration gängiger Komponenten (Datenbank, Web-Server, Sicherheit)

- Reduzierte Boilerplate-Konfiguration, also redundanten, standardmäßigen Code und XML-Einrichtungen, die in reinen Spring Framework-Anwendungen notwendig waren
- Konsistente Entwicklungserfahrung im gesamten Spring-Ökosystem

Unterstützte Technologien und Laufzeitkomponenten

Spring Boot integriert viele ausgereifte Bibliotheken und Starter-Abhängigkeiten, die für typische Backend-Anforderungen relevant sind. Diese sind zum Beispiel:

- **Spring Web** für leistungsfähige REST APIs und Web-Services
- **Spring Data JPA** zur einfachen Datenbankzugriffsschicht mit Unterstützung für relationale Systeme
- **Spring Security** für Authentifizierung, Autorisierung und rollenbasierte Zugriffssteuerung
- **Actuator** Monitoring- und Management-Endpunkte für Produktion und Betrieb

Dadurch eignet sich Spring Boot sehr gut für Backend-Anwendungen, die stark auf Integrationen, Sicherheit und robuste Betriebsfähigkeit angewiesen sind.

Entwicklungsunterstützung und Produktivität

Spring Boot ist auf Entwicklerproduktivität ausgelegt. Die wichtigsten Mechanismen dafür umfassen:

- **Spring Initializr** ein webbasiertes Tool zur Generierung von Starter-Projekten [5]
- **Automatische Abhängigkeitsverwaltung** über Starter-POMs und Bill of Materials (BOM)
- **Testunterstützung** umfangreiche Testframework-Integration für Unit- und Integrationstests
- **IDE-Integration** moderne IDEs wie IntelliJ IDEA oder Eclipse bieten tiefgreifende Unterstützung für Spring-Projekte

Dies führt zu einem schnellen Onboarding neuer Entwickler:innen und einer hohen Entwicklungsgeschwindigkeit im Team.

Eignung für die im Projekt entwickelte Lernplattform

Vorteile für die Lernplattform	Einschränkungen
Sehr große Community und umfassende Dokumentation	Höherer Ressourcenverbrauch im Vergleich zu leichteren Frameworks
Umfangreiche Standardkomponenten für REST, Sicherheit	Längere Startzeiten als kompaktere JVM-Frameworks
Gute Integration mit Datenbanken und Monitoring	Mehr Konfigurationsoptionen bedeuten potentiell mehr Komplexität
Starke IDE- und Tool-Unterstützung	Einarbeitungszeit für Spring-Konzepte bei Neueinsteigern

Tabelle 2: Zusammenfassung der Bewertung von Spring Boot für die Lernplattform

Kriterienbewertung Spring Boot

Die nachfolgende stichpunktartige Übersicht fasst die Bewertung von Spring Boot anhand der definierten Vergleichskriterien zusammen:

- **Performance und Ressourcenverbrauch:** schlecht (höherer Ressourcenverbrauch, längere Startzeiten)
- **Entwicklungsproduktivität:** sehr gut (Spring Initializr, automatische Konfiguration, gute IDE-Unterstützung)
- **Integration mit Datenbanken:** sehr gut (Spring Data JPA, Monitoring, robust)
- **Unterstützung für REST-APIs:** sehr gut (Spring Web)
- **Community- und Unternehmenssupport:** sehr gut (große Community, umfassende Dokumentation)
- **Skalierbarkeit und Wartbarkeit:** gut (umfangreiche Standardkomponenten, starke Tool-Unterstützung, etwas komplexer)

Node.js



Abbildung 3: Logo von Node.js

Node.js ermöglicht die Entwicklung von Webanwendungen und APIs in JavaScript auf der Serverseite. Durch sein ereignisgesteuertes Modell eignet es sich besonders für Anwendungen mit vielen gleichzeitigen Verbindungen. Deshalb ist Node.js auch

besonders populär für Web-APIs, Microservices und Echtzeitanwendungen wie Chat-Systeme. [6]

Somit lässt sich Node.js gut mit den Java-basierten Frameworks vergleichen, da es ebenfalls in modernen Webanwendungen und Microservices Verwendung findet, aber auf einer ganz anderen Laufzeitumgebung basiert.

Architekturprinzip: Event-driven, Non-blocking I/O

Das Ziel von Node.js ist es, viele gleichzeitige Anfragen effizient verarbeiten zu können. Dazu verwendet es ein ereignisgesteuertes, nicht-blockierendes I/O-Modell, bei dem nicht für jede Anfrage ein eigener Thread gestartet werden muss.

Dieses Prinzip führt zu folgenden Vorteilen:

- Hohe Skalierbarkeit bei I/O-lastigen Anwendungen
- Effiziente Nutzung von Ressourcen durch Single-Threaded-Event-Loop
- Gut geeignet für Echtzeit- und REST-API-Server

Unterstützte Technologien und Laufzeitkomponenten

Node.js kann mit verschiedensten Bibliotheken und Frameworks kombiniert werden, was es zu einer besonders vielseitigen Laufzeitumgebung macht. Beispiele für kombinierbare Bibliotheken und Frameworks umfassen:

- **Express.js** – beliebtes Web-Framework für REST-APIs
- **Socket.io** – Echtzeitanwendungen / Websockets
- **JWT / Passport.js** – Authentifizierung und Sicherheitsfunktionen

Durch diese Technologien ist es möglich, schnell performante REST-APIs, Echtzeitfunktionen und sichere Authentifizierungslösungen zu implementieren, was Node.js für die Lernplattform grundsätzlich geeignet macht.

Entwicklungsunterstützung und Produktivität

Neben der effizienten Verarbeitung von gleichzeitigen Anfragen unterstützt Node.js auch einen effizienten Entwicklungszyklus. Der integrierte Package-Manager **npm** vereinfacht die Installation und Verwaltung von Bibliotheken, während Hot-Reload-Tools wie *nodemon* eine schnelle Änderung am Code ermöglichen. Moderne Entwicklungsumgebungen lassen sich problemlos integrieren, wodurch Entwickler:innen direkt produktiv arbeiten können.

Eignung für die im Projekt entwickelte Lernplattform

Vorteile für die Lernplattform	Einschränkungen
Sehr populär, viele Bibliotheken und Community	Nicht JVM-basiert, keine direkte Wiederverwendung von Java-Code
Effiziente Verarbeitung von REST-Anfragen	Single-Threaded kann CPU-intensive Tasks verlangsamen
Einfach zu lernen und schnelle Entwicklungszyklen	Keine standardisierte Enterprise-Struktur, mehr Designentscheidungen nötig
Gute Unterstützung für Echtzeitanwendungen	Geringere Typensicherheit (JavaScript vs. Java)

Tabelle 3: Zusammenfassung der Bewertung von Node.js für die Lernplattform

Kriterienbewertung Node.js

Die nachfolgende stichpunktartige Übersicht fasst die Bewertung von Node.js anhand der definierten Vergleichskriterien zusammen:

- **Performance und Ressourcenverbrauch:** sehr gut bei I/O-lastigen Anwendungen, eingeschränkt bei CPU-intensiven Tasks
- **Entwicklungsproduktivität:** sehr gut (npm, Hot-Reload, schnelle Serverstarts)
- **Integration mit Datenbanken:** gut (MongoDB, SQL-Datenbanken über Bibliotheken)
- **Unterstützung für REST-APIs:** sehr gut (Express.js, Fastify)
- **Community- und Unternehmenssupport:** sehr gut (große Community, viele Tutorials)
- **Skalierbarkeit und Wartbarkeit:** in Ordnung (gut für Microservices, Typensicherheit allerdings eingeschränkt)

2.1.2 Auswahl der Backend-Technologie für die Lernplattform

Nach der Analyse der drei betrachteten Backend-Technologien **Quarkus**, **Spring Boot** und **Node.js** fällt die Wahl auf Quarkus. Im folgenden Abschnitt wird diese Entscheidung sowohl durch technische Kriterien, als auch durch praktische Erwägungen begründet.

Technische Eignung

Quarkus überzeugt vor allem durch seine sehr kurzen Startzeiten, den geringen Ressourcenverbrauch und die schnelle und effiziente Verarbeitung von REST-Anfragen. [1, 2] Besonders für Microservices oder containerisierte Deployments ist die Build-Time-Augmentation ein klarer Vorteil. Die Flexibilität, die Quarkus durch die integrierte Unterstützung gängiger Technologien wie Hibernate ORM oder RESTEasy bietet, sorgt für eine modulare, erweiterbare und zukunftssichere Systemarchitektur, die den Anforderungen einer Lernplattform mehr als nur entspricht.

Im Vergleich dazu kann man von Spring Boot zwar eine hohe Entwicklungsproduktivität und umfangreiche Standardkomponenten erwarten, es ist allerdings ressourcenintensiver, startet langsamer und erfordert für containerisiertes Deployment teilweise mehr Anpassungen. Node.js ist zwar sehr gut für I/O-lastige Anwendungen und Echtzeitanwendungen geeignet und flexibel einsetzbar, jedoch zeigt die Analyse, dass Quarkus im Sinne der Datenbankzugriffe, Sicherheit und REST-Schnittstellen besser zu den Anforderungen der Lernplattform passt. [4, 6]

Praktische Erwägungen

Ein ausschlaggebender Faktor bei der Wahl des Frameworks war auch die bisherige Erfahrung des Entwicklerteams. Quarkus wurde bereits im Unterricht behandelt und hat sich in vergangenen Projekten als zuverlässig erwiesen. Dadurch konnte auf vorhandenes Wissen zurückgegriffen werden, was den Einstieg erleichtert und die Entwicklungsgeschwindigkeit fördert. Spring Boot wurde bisher nicht im Unterricht behandelt und hätte daher eine längere Einarbeitungszeit erfordert. Node.js wurde im Unterricht in Ansätzen behandelt, allerdings nicht in einem Umfang, der für die komplexe Backend-Entwicklung der Lernplattform ausreichend wäre.

Abwägung der Backend-Frameworks

Die nachfolgende Tabelle fasst die Bewertung von Quarkus, Spring Boot und Node.js anhand der zuvor definierten Vergleichskriterien zusammen. Sie verdeutlicht, in welchen Bereichen Quarkus die beste Übereinstimmung mit den Anforderungen der Lernplattform bietet.

Kriterium	Quarkus	Spring Boot	Node.js
Performance und Ressourcenverbrauch	sehr gut	schlecht	gut
Entwicklungsproduktivität	gut	sehr gut	sehr gut
Integration mit Datenbanken	gut	sehr gut	gut
Unterstützung für REST-APIs	sehr gut	sehr gut	sehr gut
Community- und Unternehmenssupport	in Ordnung	sehr gut	sehr gut
Skalierbarkeit und Wartbarkeit	sehr gut	gut	in Ordnung

Tabelle 4: Vergleich von Quarkus, Spring Boot und Node.js (kompakt)

Fazit

Auf Basis der technischen Analyse und der praktischen Erfahrung des Teams wurde Quarkus als Backend-Framework für die Lernplattform ausgewählt. Durch diese Entscheidung ist sichergestellt, dass die Plattform sowohl performant als auch erweiterbar

bleibt und das Entwicklerteam effizient arbeiten kann. Spring Boot und Node.js bleiben zwar interessante Alternativen, werden aber nicht gleichermaßen den spezifischen Anforderungen des Projekts gerecht.

2.2 Frontend

Frontend Auswahl

2.2.1 Angular

Was ist Angular. Vor- und Nachteile usw

2.2.2 Flutter

Was ist Flutter. Vor- und Nachteile usw

2.2.3 Ergebnis

Warum wir uns für Angular entschieden haben

3 Praktische Umsetzung

3.1 Usecases

3.1.1 User Flow

Der User Flow beschreibt klar, wie der typische Interaktionsablauf eines Schülers oder einer Schülerin mit der Lernplattform aussieht. Dabei ist wichtig, welche Schritte der User von der Anmeldung bis zur Lernaktivität durchläuft und welche Handlungsmöglichkeiten ihm dabei zur Verfügung stehen. Besonders entscheidend sind hierbei die Hauptfunktionen der Anwendung, nicht aber technische Details oder Sonderfälle.

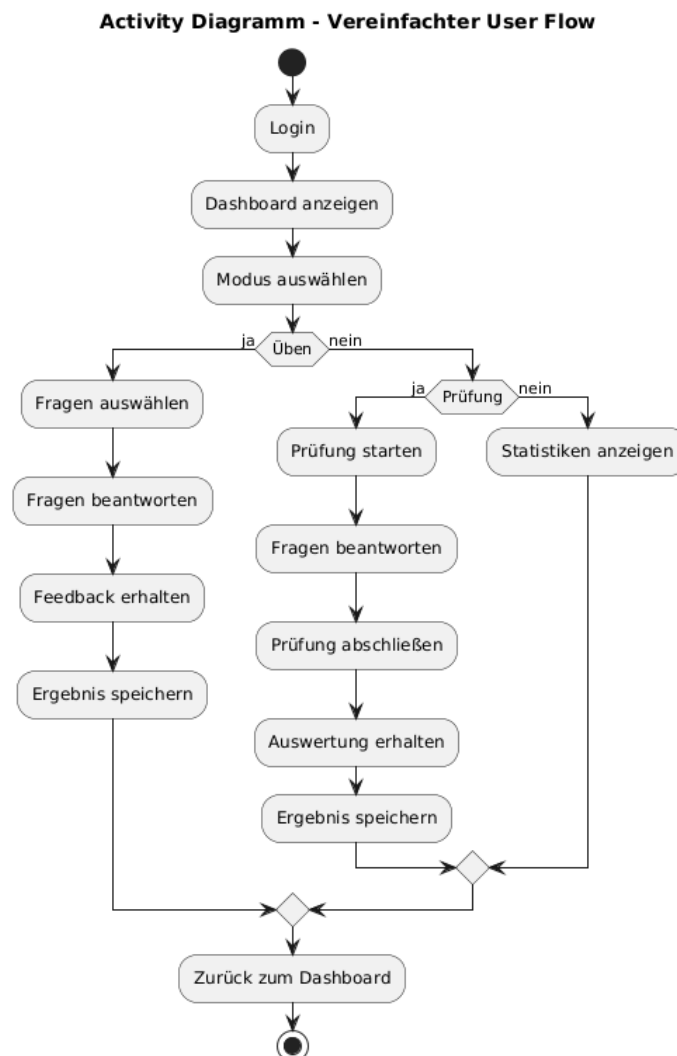


Abbildung 4: Ablaufdiagramm eines Users
[7]

Aus diesem User Flow lassen sich 3 zentrale Abläufe erkennen, die den Kernfunktionen der Lernplattform entsprechen. Sie sind die Basis für die weiteren Use Cases:

- **Üben**
Bearbeiten von Fragen mit unmittelbarem Feedback
- **Prüfungssimulation**
Bearbeiten einer Prüfungssituation mit Auswertung am Ende
- **Fortschrittsübersicht**
Einsehen von Lernstatistiken und gespeicherten Ergebnissen

Diese drei Pfade werden in den folgenden Abschnitten als eigenständige Use Cases detailliert beschrieben.

3.1.2 Übungsmodus

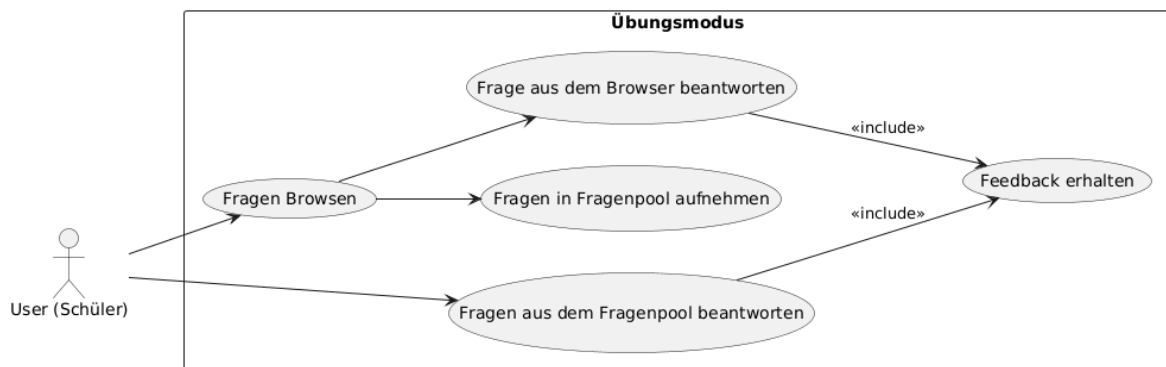


Abbildung 5: Use-Case Diagramm: Übungsmodus

[7]

Der Übungsmodus ist einer der Kernfunktionen der Lernplattform und soll Schüler:innen die Möglichkeit bieten, gezielt für Prüfungssituationen lernen zu können. Der Ablauf von diesem Modus beginnt normalerweise damit, dass der User durch vorhandene Fragen von anderen Nutzern stöbert. Dabei kann er Themenbereiche auswählen, dessen Fragen betrachten und sich so einen Überblick über den Umfang des Themas beschaffen.

Während des Browsens besteht die Möglichkeit, interessante Fragen in seinen eigenen Fragenpool aufzunehmen, um sie später gezielt zu lernen. Der primäre Anwendungszweck des Übungsmodus besteht darin, wiederholt Fragen aus dem eigenen Fragenpool zu bearbeiten. Alternativ können Fragen direkt beim Browsen beantwortet werden, wobei das System diese sofort auswertet und eine Rückmeldung über die Richtigkeit der Antworten gibt. Somit können Nutzer direkt ihre Fehler erkennen und korrigieren.

Wenn der Nutzer Fragen in seinen eigenen Fragenpool aufgenommen hat, kann direkt aus dem Fragenpool gelernt werden. Auch hier werden Antworten ausgewertet und direktes Feedback gegeben, im Unterschied zum direkten Beantworten während des Browsens wird im Fragenpool allerdings gespeichert, ob eine Frage zuletzt richtig oder falsch beantwortet wurde. Auf diese Weise kann das System die in Kapitel 1.4 analysierte Lernstrategie erfolgreich anwenden und den Nutzer effektiv beim Lernvorgang unterstützen.

Der Übungsmodus bietet somit einen strukturierten Ablauf, bei dem selbständiges Lernen unterstützt, die Wiederholung wichtiger Inhalte erleichtert, und durch unmittelbares Feedback der Lernprozess effektiv begleitet wird. Anders als beim Prüfungsmodus steht hier also nicht die Bewertung, sondern der Lernprozess im Vordergrund. Aus diesem Grund erfolgt das Feedback auch unmittelbar nach jeder beantworteten Frage.

Nach jedem Übungsdurchlauf verfügt der User über eine Rückmeldung zu seinen Antworten und kann leicht erkennen, welche Themen bereits sicher beherrscht werden und in welchen Bereichen mehr Übung notwendig ist.

3.1.3 Prüfungssimulationen

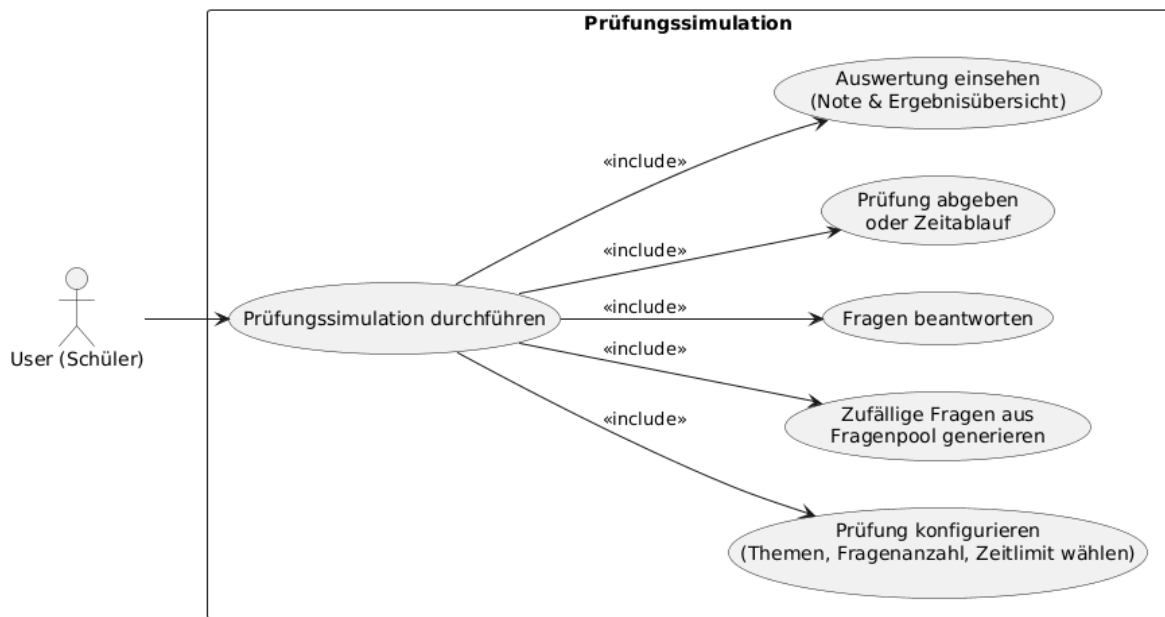


Abbildung 6: Use-Case Diagramm: Prüfungsmodus

[7]

Nachdem die Schüler:innen im Übungsmodus ihr Wissen vertiefen konnten und einige Themen wiederholt haben, dient der Prüfungsmodus dazu, dieses Wissen unter realitätsnahen Bedingungen unter Beweis zu stellen. Er ermöglicht es, den aktuellen Wissensstand zu testen und ein Gefühl für echte Prüfungssituationen zu bekommen.

Im Gegensatz zum Übungsmodus steht im Prüfungsmodus nicht der Lernprozess, sondern die Simulation einer echten Prüfungssituation im Vordergrund. Dazu kann der User individuelle Prüfungskonfigurationen vornehmen, indem er die zu testenden Themenbereiche auswählt, sowie die Anzahl der Fragen und das gewünschte Zeitlimit bestimmt. Auf diese Weise kann die Prüfung sowohl in Umfang als auch in Schwierigkeit auf den aktuellen persönlichen Lernfortschritt angepasst werden.

Ist der Nutzer mit seiner vorgenommenen Konfiguration für seine Prüfung zufrieden, so werden zunächst zufällig ausgewählte Fragen aus dem persönlichen Fragenpool zu einer Prüfungssimulation zusammengestellt. Um das Gefühl einer echten Prüfung zu stärken hat der User während der Prüfung zu jeder Zeit die Möglichkeit, zwischen allen Fragen zu navigieren und seine Fragen beliebig oft zu ändern. Feedback erhält er aber erst nach eigener Abgabe oder nach Ablauf des Zeitlimits, um die Prüfungssituation möglichst authentisch nachzubilden.

Im Anschluss erfolgt die Auswertung. Der User erhält nun eine zusammengefasste Rückmeldung über seine Leistung. Es wird das Prüfungsergebnis und eine Übersicht über richtig und falsch beantwortete Fragen übermittelt. Wichtig dabei ist, dass die Fragen einer Prüfung hierbei unabhängig vom Übungsmodus behandelt werden und nicht in die Lernstrategie oder den Fortschritt des Fragenpools einfließen. Der Prüfungsmodus dient also ausschließlich der Überprüfung des Wissenstands. Dadurch bleiben Prüfungs- und Übungsmodus klar voneinander getrennt.

Die Ergebnisse vergangener Prüfungen werden gespeichert und können im Zuge der Fortschritts- und Statistikübersicht später erneut eingesehen werden. Somit ist es für den User möglich, den eigenen Leistungsfortschritt über mehrere Prüfungsversuche hinweg nachzuverfolgen, ohne den Lernprozess im Übungsmodus zu beeinflussen.

3.1.4 Fortschrittsübersicht

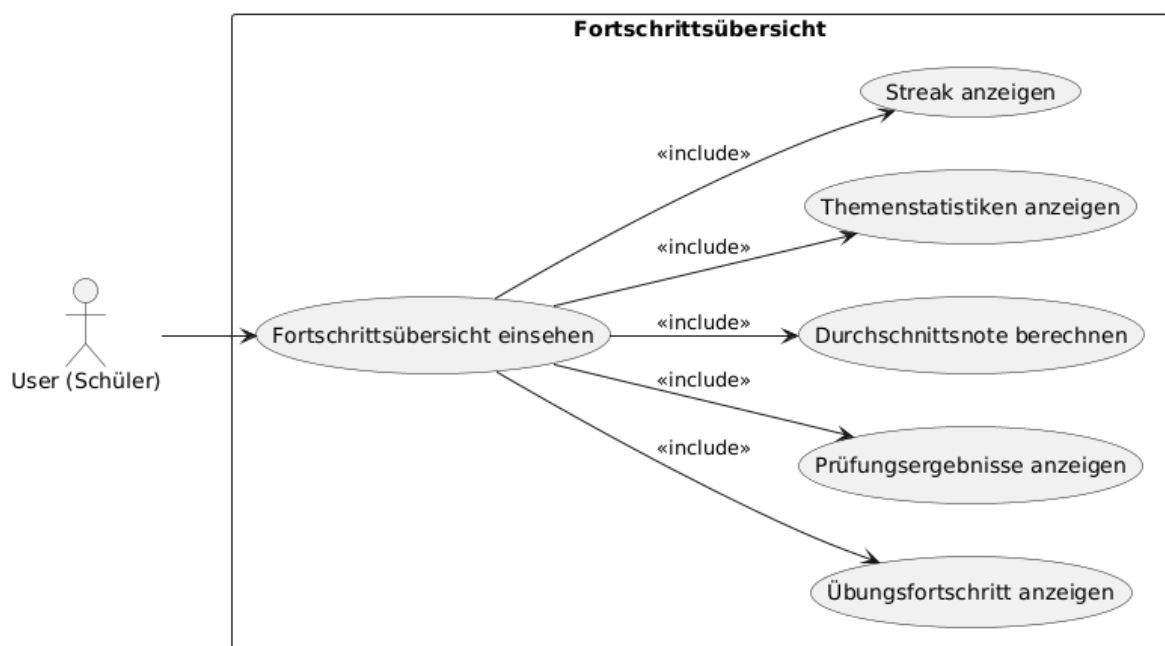


Abbildung 7: Use-Case Diagramm: Fortschrittsübersicht
[7]

User (Schüler) möchte seinen Lernfortschritt jederzeit einsehen und Statistiken betrachten. Usecase-Diagramme für User (Schüler) → Fortschritt (% der unbeantworteten, falsch beantworteten, 1x, 2x, genug beantworteten Fragen), Durchschnittswertung bei Prüfungen usw.

3.2 Use Case: Üben

Der Kern der Anwendung ist der Übungsmodus, welcher Schüler:innen ermöglicht, gezielt Maturafragen zu trainieren und ihr Wissen Stück für Stück auszubauen. Anders als beim Prüfungsmodus steht hier nicht der Leistungsdruck im Vordergrund, sondern das aktive Lernen mit direktem Feedback. Nutzer:innen soll es ermöglicht werden, ein Fundament für ihr Wissen festigen zu können.

Funktionen umfassen das Auswählen von bestimmten Themen, das Zusammenstellen von eigenen Fragenpools und das Erstellen und Einsehen von Lösungswegen für die jeweiligen Aufgaben. Nutzer:innen können hierbei also genau die Themen lernen, die ihnen Probleme bereiten, und das durch eigene Fragen oder Fragen anderer Nutzer:innen.

Die Plattform unterstützt dabei verschiedene Fragetypen, um das Lernerlebnis flexibler zu gestalten. Um nicht einfach nur ein Ergebnis von "richtig" und "falsch" zu liefern, können Nutzer:innen auch Lösungswege einsehen, um das Verständnis nachhaltig zu fördern.

Durch diese wiederholbare und flexible Übungsumgebung eignet sich die Anwendung sowohl für den schnellen Gebrauch als Wissenscheck als auch für die langfristige Vorbereitung auf Tests, Schularbeiten oder die Matura.

3.2.1 Verwaltung von Fragenpools

Auswahl eigener Fragen, Erstellung von individuellen Pools, Backend- und Frontend-Umsetzung.

3.2.2 Fragenhandling und Fragetypen

Speicherung, Abruf, Darstellung verschiedener Fragetypen, Feedbacksystem.

Wie in Abbildung 9 dargestellt befindet sich der/die Nutzer:in nun beim Question-Runner, also der Ansicht, bei der Fragen beantwortet und ausgewertet werden können. Hierbei gibt es 3 verschiedene Modi, in der sich der Question-Runner befinden kann:

- **Übungsmodus** (3.2.3): Nutzer:in erhält direkt Feedback zu den Antworten, ohne Zeitdruck.
- **Prüfungsmodus** (3.3.1): Prüfung wird unter Zeitlimit simuliert, Feedback erfolgt erst am Ende.
- **Reviewmodus** (3.3.2): Bereits abgeschlossene Tests können erneut durchgesehen werden, Antworten und Lösungswege sind sichtbar, Änderungen sind nicht möglich.

User-Interface des Question Runners

Üben > Fragen beantworten

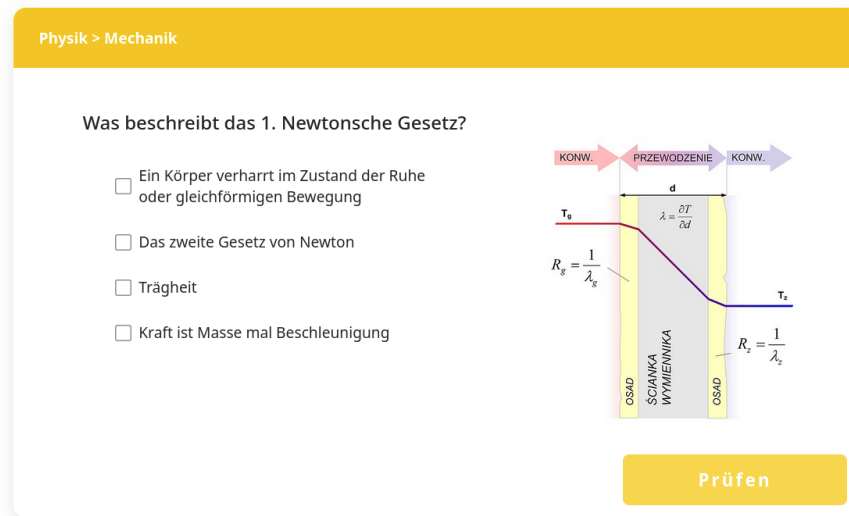


Abbildung 8: Beispielhafte Ansicht einer Frage im Question Runner

Der Question Runner zeigt zunächst, aus welchem Fach und welchem Themenpool die Frage stammt, die der/die Nutzer:in gerade bearbeitet. Direkt darunter wird die Frage als Überschrift angezeigt, rechts daneben ggf. das zugehörige Bild.

Unter der Frage befinden sich die Antwortfelder, abhängig vom Fragetyp: Multiple-Choice-Fragen als Auswahlfelder (Abbildung 9a), Freitextfragen als Eingabefeld (Abbildung 9b). Ganz unten befinden sich die Aktionsbuttons: im Übungsmodus der „**Prüfen**“-Button, in den anderen Modi allgemeine Navigationsbuttons (**Weiter/Zurück**).

Datenfluss im Question Runner

Im Übungsmodus ruft der Question Runner zunächst die angegebene Frage anhand ihrer eindeutigen ID ab, falls der/die Nutzer:in eine konkrete Frage auswählen möchte, wie bei der Verwaltung von Fragenpools (3.2.1) beschrieben.

Wurde keine einzelne Frage ausgewählt, werden zunächst die Filterkriterien für Themen und Fragetypen geprüft. Anschließend werden die passenden Fragen aus dem Fragenpool des/der Nutzers/Nutzerin abgerufen und dem Question Runner übergeben. Auf diese Weise wird sichergestellt, dass sowohl gezieltes Üben einzelner Fragen als auch das Bearbeiten thematisch gefilterter Mengen möglich ist.

Der Question Runner erhält von anderen Komponenten im Frontend eine **Liste von Frage-IDs** oder alternativ eine einzelne ID, die der/die Nutzer:in bearbeiten möchte. Diese IDs repräsentieren die Fragen, die im Anschluss nacheinander abgerufen und angezeigt werden. Die Auswahl der IDs erfolgt zuvor in der Themen- oder Fragenpool-Ansicht und wird hier nicht weiter gefiltert.

Das Backend liefert das vollständige Frageobjekt zurück, das alle für die Anzeige im Question Runner erforderlichen Informationen enthält (siehe Abschnitt „User-Interface des Question Runners“).

Nachdem der/die Nutzer:in eine Antwort eingibt, hängt das Verhalten vom Modus ab:

- **Übungsmodus:** Die Antwort wird direkt ans Backend gesendet und dort ausgewertet. Das Ergebnis wird sofort zurückgesendet und im Question Runner angezeigt. Danach ruft der Question Runner automatisch die nächste ID auf.
- **Prüfungsmodus:** Antworten werden gesammelt und erst nach Abschluss der Prüfung ans Backend gesendet. Feedback erfolgt danach.
- **Reviewmodus:** Nur Anzeige von Antworten und Lösungswegen. Der/Die Nutzer:in kann hier also keine Antworten eingeben.

Unterstützung verschiedener Fragetypen

Neben der Unterscheidung der Modi werden im Question Runner die verschiedenen Fragetypen berücksichtigt. Aktuell werden zwei Haupttypen unterstützt:

- **Multiple-Choice-Fragen:** Nutzer:innen wählen eine oder mehrere richtige Antworten aus einer Liste vorgegebener Auswahlmöglichkeiten aus. Multiple-Choice-Fragen sind besonders geeignet, um Faktenwissen abzufragen oder schnelle Wissenskontrollen durchzuführen. Das System wertet die Antwort automatisch aus und kann im Übungsmodus direkt Feedback geben. Die Darstellung erfolgt als übersichtliche Auswahlfelder, die klar zwischen ausgewählt und nicht ausgewählt unterscheiden. (siehe Abbildung 9a)
- **Freitext-Fragen:** Nutzer:innen geben ihre Antwort in ein Texteingabefeld ein. Dieser Fragetyp eignet sich für offene Aufgabenstellungen, bei denen die Antwort nicht auf eine vorgegebene Auswahl beschränkt ist. Im Backend wird die Eingabe des/der Nutzer:in mit einer Menge möglicher korrekter Antworten abgeglichen. Entspricht die Eingabe einer dieser Antwortmöglichkeiten, wird die Antwort als korrekt bewertet, andernfalls als falsch. Im Frontend steht ein ausreichend großes Eingabefeld bereit, um längere Antworten komfortabel einzugeben (siehe Abbildung 9b).

Üben > Fragen beantworten

(a) Multiple-Choice-Frage

Üben > Fragen beantworten

(b) Freitext-Frage

Abbildung 9: Question-Runner: Beispiele für verschiedene Fragetypen

3.2.3 Übungsmodus mit direktem Feedback

Technische Umsetzung, Ablauf, User Experience.

3.2.4 Lösungswege und Nutzerinteraktion

Einsehen von Lösungswegen, Feedbackmechanismen

3.3 Use Case: Prüfungssimulation

3.3.1 Prüfungsmodus mit Zeitlimit

Simulation der Prüfung, Zeitlimit, realistische Prüfungssituation.

3.3.2 Bewertung und Ergebnisausgabe

Ergebnisanzeige am Ende, Notenberechnung, Auswertung.

3.4 Use Case: Fortschritt einsehen & analysieren

Ein wichtiger Bestandteil unserer Lernplattform ist Fortschrittseinsicht, um den User die Möglichkeit zu bieten seinen aktuellen Wissensstand jeder Zeit einsehen zu können. Mit übersichtlichen Diagrammen und Statistiken hat der User immer einen klaren Überblick auf seinen Lernfortschritt und kann so effizient einschätzen wie viel Übungsbedarf besteht. Filteroptionen ermöglichen es auch nach verschiedenen Themenpools zu filtern damit Schüler gezielt erkennen in welchen Themenpools sie noch mehr Übung bräuchten.

Neben der Darstellung des Lernfortschritts mit Diagrammen sind Statistiken auch zur Motivation gut geeignet, ein sichtbarer Fortschritt motiviert für zukünftiges Lernen. Ein positives Lernerlebnis kann Nutzer auch für weitere Lernziele anspornen bzw. ihnen helfen regelmäßiger zu üben um ihre Leistung zu verbessern.

Aufgebaut sind die Statistiken und Diagramme auf den Ergebnissen und Daten aus dem Übungsmodus oder wenn der User eine Frage explizit übt. Die Auswahl der gestellten Fragen erfolgt über ein recherchiertes und ausgewähltes Lernsystem, um den Wissenstand langfristig im Kopf zu behalten

3.4.1 Lernstrategien und Fehleranalyse

Identifikation von Wiederholungsbedarf und Schwachstellen.

Um den Lernerfolg langfristig zu verbessern, ist es notwendig, das Lernverhalten analysieren und Schwachstellen zu erkennen. Auf Grundlage der Fehleranalyse wird der Wiederholungsbedarf festgelegt. Da Nutzer:innen verstärkt die Fragen oder Themenpools üben die ihnen schwer fallen wird ihr Wissen und Sicherheit gestützt. Aufgaben und Fragen die sie bereits beherrschen und mehrfach korrekt beantwortet hatten, kommen hingegen in selteneren Abständen bzw. fallen zur gänze raus. Durch diese Anpassungen wird effizienter gelernt und somit auch mehr Lernerfolg erzielt.

Darüber hinaus unterstützt diese Funktion das selbstreflektierende Lernen. Die Nutzer:innen können nachvollziehen und selbst Muster erkennen, bei welchen Inhalten oder Fragen sie des öfteren mal Schwierigkeiten haben. Dadurch können die User selbst sehen wo sie Probleme haben und besser ihre Personalisierten Fragenpools auswählen, diese können sie anschließend üben.

3.4.2 Spaced Repetition nach Leitner

Technische Umsetzung der Lernstrategie, Speicherintervalle, Fortschrittslevel.

Die Grundlage für die Lernplattform bildet das Leitner-System in Kombination mit Spaced Repetition, die in Kapitel 1.5 beschrieben wurden. Die Lernmethoden bestehen aus 3 wichtigen Punkten

1. **Level:** Fragen werden in Level eingeteilt - je öfter richtig desto höher das Level
2. **Wiederholungsintervall:** Fragen werden gesperrt wenn man sie richtig hatte und erst nach einem gewissen Zeitraum entsperrt
3. **Anzeige:** Gesperrte und offene Fragen werden angezeigt Die Anzeige im Frontend dient dazu den

In Abbildung 4 Homekomponente bzw. die Umsetzung der Lernstrategien sieht man die verschiedenen "Level". Die Level sind nach dem Leitner-System aufgebaut, von denen es in diesem Fall vier gibt: Startlevel, Basislevel, Trainingslevel und Highscorelevel.

Jede Frage wird durch QuestionPoolEntry abgebildet, diese speichert alle relevante Daten für die Fehleranalyse oder das Wiederholen. Für das einteilen in die Level sind die Einträge lastAnsweredCorrectly und correctCount wichtig. LastAnsweredCorrectly merkt sich ob die Frage das letzte mal korrekt beantwortet wurden und correctCount wie oft du diese Frage bereits richtig hattest. Mithilfe einer Abfrage kann eingeordnet werden welche Frage in welches Level kommt. Es verläuft nach dem Schema

- **Startlevel:** Dort sind die falsch oder nicht beantworteten Fragen, diese sind immer zum üben offen.
- **Basislevel:** Hier befinden sich Fragen die einmalig richtig beantwortet waren, direkt danach sind sie für 1 Tag gesperrt.
- **Trainingslevel:** Fragen die der User 2x richtig hatte kommen ins Trainingslevel und sind nun 3 Tage gesperrt.
- **Highscorelevel:** Hier befinden sich Fragen die schon zur Genüge geübt wurden und somit kaum mehr geübt werden müssen.

Das Sperren erfolgt durch die Auswertung des Parameters answeredAt. So lässt sich nachvollziehen, wann der Nutzer die Frage zuletzt bearbeitet hat, ob die Antwort korrekt oder falsch war, in welchem Level sich die Frage zuvor befand bzw. wie hoch der correctCount ist. Schlussendlich lässt sich dann sagen wie lange die Frist zum Wiederholen eingestellt wird.

Mit einem Klick auf üben werden nicht nur irgendwelche Fragen ausgewählt, sondern genau die die der User am dringendsten benötigt. Sozusagen die falsch oder nicht beantworteten Fragen zuerst, darauffolgend erst die mehrmals korrekt beantworteten Inhalte.

Die Anzeige im Frontend dient dazu den Nutzer:innen einen Überblick über ihren eigenen Lernfortschritt zu geben. Die einzelnen Level im Leitner-System werden als Progressbars visualisiert und mit Farben wird dem User sein Fortschritt symbolisiert.

3.4.3 Lernstatistiken und Visualisierung

Erhebung und grafische Darstellung der Daten zur Nutzerleistung. Bibliotheken und Darstellungskonzepte.

3.5 Usability-Test mit Mitschülern

Testbeschreibung, Szenarien, Feedback, Umsetzung von Verbesserungen.

3.6 Implementierungsdetails

3.6.1 Datenmodell

Datenbanktabellen

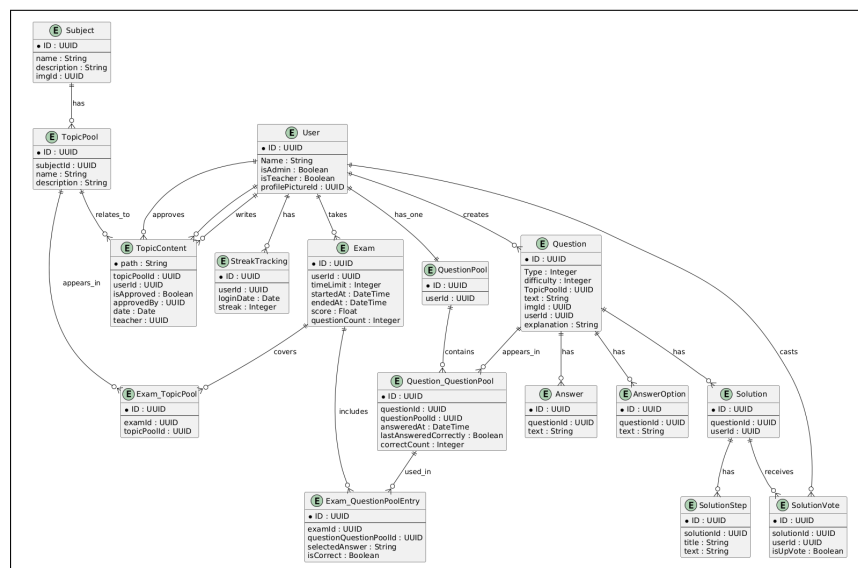


Abbildung 10: Datenbankmodell der Anwendung

3.6.2 Überblick über Komponenten

Question Runner - wird fast überall benutzt

3.6.3 Custom Css-Klassen und eingebundene Bibliotheken

Css Klassen, welche öfter benutzt werden - Bibliotheken, wie Statistik dingens

4 Zusammenfassung

Literaturverzeichnis

- [1] Red Hat, „Quarkus — Supersonic Subatomic Java,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://quarkus.io>
- [2] —, „Quarkus Guide – Re-augment a Quarkus Application,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://quarkus.io/guides/reaugmentation>
- [3] —, „Quarkus - Extensions,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://quarkus.io/extensions/>
- [4] Pivotal Software, Inc., „Spring Boot,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://docs.spring.io/spring-boot/>
- [5] —, „Spring Initializr,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://start.spring.io/>
- [6] OpenJS Foundation, „Node.js,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://nodejs.org/en>
- [7] Arnaud Roques, „PlantUML,” letzter Zugriff am 05.01.2026. Online verfügbar: <https://www.plantuml.com/plantuml/uml/>

Abbildungsverzeichnis

1	Logo von Quarkus	5
2	Logo von Springboot	7
3	Logo von Node.js	9
4	Ablaufdiagramm eines Users	14
5	Use-Case Diagramm: Übungsmodus	15
6	Use-Case Diagramm: Prüfungsmodus	16
7	Use-Case Diagramm: Fortschrittsübersicht	17
8	Beispielhafte Ansicht einer Frage im Question Runner	19
9	Question-Runner: Beispiele für verschiedene Fragetypen	20
10	Datenbankmodell der Anwendung	23

Tabellenverzeichnis

1	Zusammenfassung der Bewertung von Quarkus für die Lernplattform .	6
2	Zusammenfassung der Bewertung von Spring Boot für die Lernplattform	9
3	Zusammenfassung der Bewertung von Node.js für die Lernplattform . .	11
4	Vergleich von Quarkus, Spring Boot und Node.js (kompakt)	12

Quellcodeverzeichnis

Anhang